Principles of model based clustering in practice

Lieven Clement

Invalid Date

Table of contents

1 Agglomerative hierarchical clustering based on the classification likelihood **1**
1.1 Classification likelihood with $\Sigma_k = \lambda I$ . . . 2
1.2 Ward's method . . . 3
1.3 Example . . . 4
1.4 Full cluster trees . . . 4

2 EM based iterative relocation using the mixture likelihood **7**
2.1 Example of model based clustering with two groups . . . 7
2.2 Simulate toy Data . . . 8
2.3 Parameter estimation . . . 10
2.3.1 EM algorithm . . . 11
2.4 Example . . . 12
2.4.1 Initialize . . . 12
2.4.2 EM algorithm . . . 14

3 From EM to CEM: link with k-means **20**
3.1 Soft versus hard assignment . . . 20
3.2 CEM with spherical clusters with equal variance and equal proportions . . . 20
3.3 Implicit assumptions of k-means . . . 21
3.4 Example . . . 21

Session info **23**

1 Agglomerative hierarchical clustering based on the classification likelihood

Fraley and Raftery (1998) state that

"The classification likelihood can be used as the basis for agglomerative hierarchical clustering"

"This approach subsumes several earlier proposals based on Gaussian mixtures: $\Sigma_k = \lambda I$ gives the sum of squares criterion, long known as a heuristic (Ward), in which clusters are spherical and have equal volumes."

1.1 Classification likelihood with $\Sigma_k = \lambda I$

Model based hierarchical clustering merges at each stage the pair of clusters that maximises the resulting classification likelihood

$$L_C(\theta_1, \dots, \theta_G; \gamma_1, \dots, \gamma_n | \mathbf{X}) = \prod_{i=1}^{n} f_{\gamma_i}(\mathbf{x}_i | \theta_{\gamma_i}),$$

with $\gamma_i = k$ if observation i belongs to cluster k.

With $\Sigma_k = \lambda I$, the density of a p-dimensional observation in cluster k is

$$f_k(\mathbf{x}_i) = (2\pi\lambda)^{-p/2} \exp\left(-\frac{\|\mathbf{x}_i - \mu_k\|^2}{2\lambda}\right)$$

and the log classification likelihood becomes

$$\log L_C = -\frac{np}{2}\log(2\pi\lambda) - \frac{1}{2\lambda}\sum_{k=1}^{G}\sum_{i:\gamma_i=k}\|\mathbf{x}_i - \mu_k\|^2$$

- The maximum likelihood estimate of μ_k is the cluster mean.
- The double sum then becomes the within-cluster sum of squares W, i.e. the total squared distance of all observations to the mean of their own cluster.
- The maximum likelihood estimate of λ is $\widehat{\lambda} = W/(np)$.

Plugging in these estimates gives

$$\log L_C = -\frac{np}{2}\log\left(\frac{2\pi W}{np}\right) - \frac{np}{2}$$

- The only term that depends on the partition is W.
- Hence, maximising the classification likelihood is equivalent to minimising the within-cluster sum of squares.

1.2 Ward's method

- Ward's hierarchical clustering merges at each stage the two clusters that lead to the smallest increase of the within-cluster sum of squares W.
- Model based hierarchical clustering with $\Sigma_k = \lambda I$ merges the two clusters that lead to the smallest decrease of the classification likelihood.
- Both should make the same choice at each stage and produce the same tree!
- Ward proposed his criterion as a heuristic: keep the clusters as compact as possible.
- Model based clustering reveals the model that is implicitly assumed:

Part of the model	Implication for the clusters
I (identity matrix)	spherical: the same spread in every direction
one λ for all clusters	equal volume: all clusters have the same size

- Model based clustering allows us to relax these assumptions with the other parametrisations of Σ_k (see Table 1 in Fraley and Raftery, 1998), e.g. clusters with different volumes (λ_k), elongated clusters or clusters with a different orientation.
- Note, that k-means minimises the same criterion W; only the search strategy differs:

	Criterion	Strategy
Ward	minimise W	agglomerative: merge clusters step by step
k-means	minimise W	relocation: move observations between clusters (CEM)
Model based with $\Sigma_k = \lambda I$	maximise L_C, i.e. minimise W	agglomerative or relocation

- So Ward and k-means are two algorithms for the same model: spherical clusters with equal volume.
- The link between k-means and EM based relocation is explained in the section on CEM below.

1.3 Example

We simulate bivariate data with two groups and compare model based hierarchical clustering with the spherical equal volume model (`"EII"` in `mclust`) with Ward's method.

```
library(mclust)

set.seed(1)
X <- rbind(
  matrix(rnorm(60, mean = 0), ncol = 2),
  matrix(rnorm(60, mean = 4), ncol = 2)
  )
```

1.4 Full cluster trees

To compare the full trees, we use the same height for both dendrograms: the within-cluster sum of squares W of the partition after each merge.

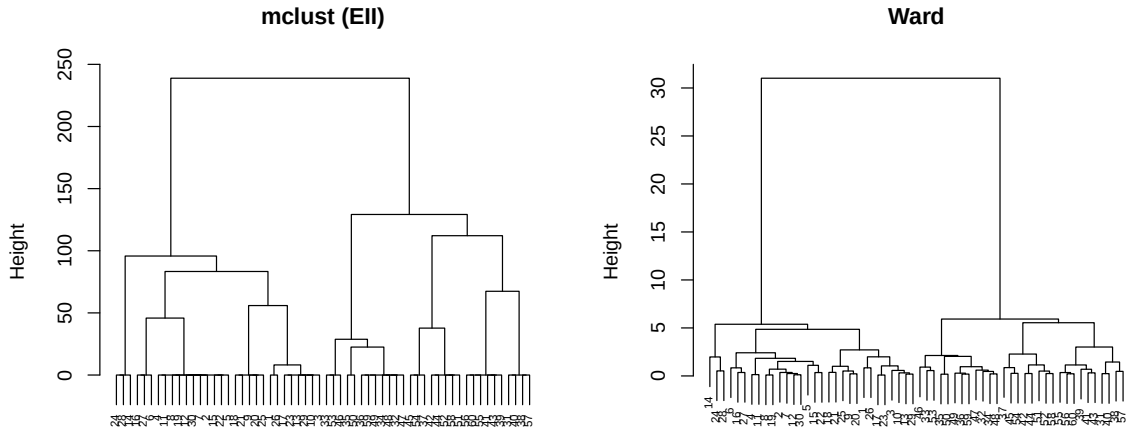
- For Ward's method, we obtain the partitions by cutting the tree.
- For model based hierarchical clustering, `hclass()` returns the partitions for every number of clusters. Note, that `hclass()` returns them in increasing order of G, while the merges in the tree are ordered from $G = n - 1$ to $G = 1$.

```
## within-cluster sum of squares of a partition
n <- nrow(X)

## model based hierarchical clustering with spherical equal volume model
hcEIIobj <- hc(X, modelName = "EII", use = "VARS")
treeEII <- as.hclust(hcEIIobj)
treeEII$labels <- NULL


## Ward's method
treeWard <- hclust(dist(X), method = "ward.D2")


par(mfrow = c(1, 2))
plot(treeEII, main = "mclust (EII)",
     xlab = "", sub = "", cex = 0.6)
plot(treeWard, main = "Ward",
     xlab = "", sub = "", cex = 0.6)
```



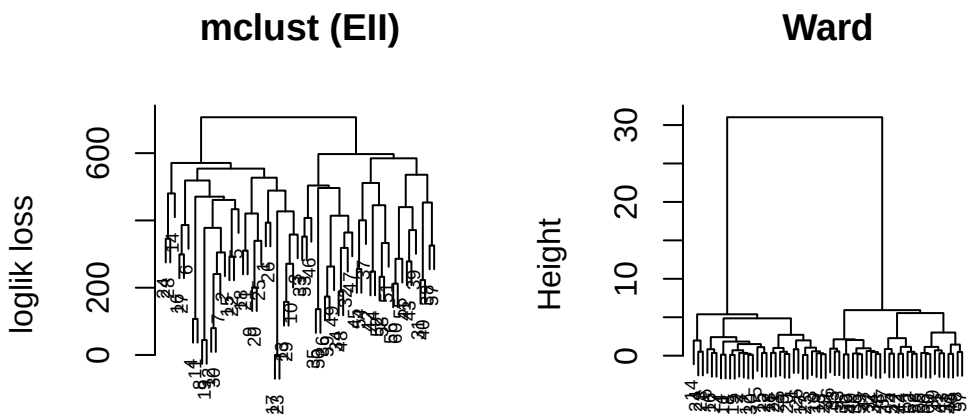
- The clustering is the same, but looks different because the loss in log-likehood gets at NA when too many clusters are used.
- This is an anomaly of the software as the log-likelihood is calculated separately for each cluster. So it does not use the model specification!
- Calculating the loglik ourselves:

```
## within-cluster sum of squares of a partition
withinSS <- function(cl, dat)
  sum(sapply(unique(cl), function(k)
    sum(scale(dat[cl == k, , drop = FALSE], scale = FALSE)^2)))

n <- nrow(X)
p <- ncol(X)
W <- hclass(hcEIIobj, G = 1:(n - 1)) |>
  apply(2, withinSS, dat = X)                    # withinSS from em.qmd
logLik <- -n * p / 2 * log(2 * pi * W / (n * p)) - n * p / 2

treeEII$height <- rev(max(logLik) - logLik)        # loss of log-likelihood, from G = n-1 dow

par(mfrow = c(1, 2))
plot(treeEII, main = "mclust (EII)",
     xlab = "", ylab="loglik loss", sub = "", cex = 0.6)
plot(treeWard, main = "Ward",
     xlab = "", sub = "", cex = 0.6)
```

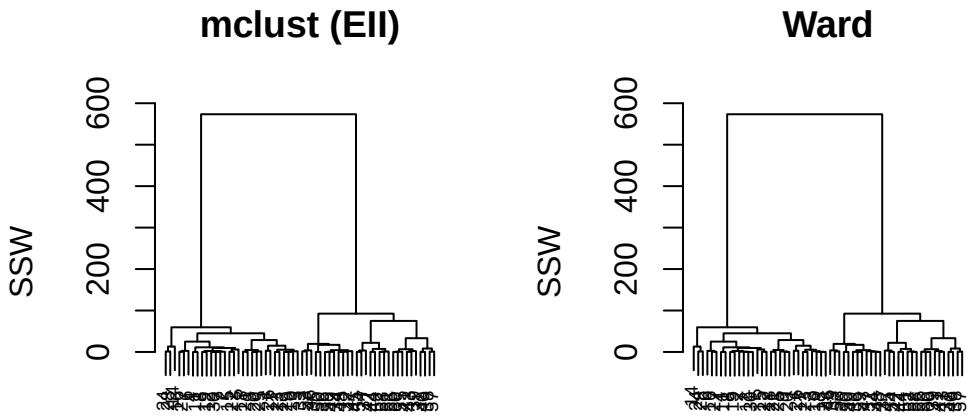


- It is still difficult to compare the clustering.
- We will now redefine the height by using the same metric, i.e. the within sum of squares (Ward metric)

```
## model based hierarchical clustering with spherical equal volume model
treeEII$height <- hclass(hcEIIobj, G = 1:(n - 1)) |>
  apply(2, withinSS, dat = X) |>
  rev()

## Ward's method
treeWard$height <- sapply((n - 1):1, function(g) withinSS(cutree(treeWard, g), dat = X))

par(mfrow = c(1, 2))
plot(treeEII, main = "mclust (EII)",
     xlab = "", sub = "", ylab = "SSW", cex = 0.6)
plot(treeWard, main = "Ward",
     xlab = "", sub = "", ylab = "SSW", cex = 0.6)
```



- Both trees merge the same clusters at the same height.
- The order of the leaves can differ slightly, because the two branches below a merge can be drawn in either order.

2 EM based iterative relocation using the mixture likelihood

2.1 Example of model based clustering with two groups

We will use a toy example to explain the EM algorithm for model based clustering.

- Two groups $k = 1, 2$
- Univariate observations are observed x_i with $i = 1, \ldots, n$
- The data of group j follows a normal distribution $N(u_j, 1)$ with a variance $\sigma^2 = 1$ and a mean μ_j that depends on the group $j = 1, 2$.

The data follows the following mixture distribution:

$$f(x) = \pi_1 f_1(x) + (1 - \pi_1) f_2(x)$$

With

- π_1 the probability that a random sample from the population belongs to group 1

- $f_1(x)$ the density of the data in group 1, i.e. $N(\mu_1, 1)$
- $f_2(x)$ the density of the data in group 2, i.e. $N(\mu_2, 1)$.
- Unknowns? Group membership z_i, the group means μ_j and the proportion of subjects in the population of group 1 π_1 are unknown.

We can estimate the model parameters $\theta = [\mu_1, \mu_2, \pi_1]^T$ using maximum likelihood.

$$L(\mu_1, \mu_2, \pi_1 | \mathbf{X}) = \prod_{i=1}^{n} [\pi_1 f_1(x_i) + (1 - \pi_1) f_2(x_i)]$$

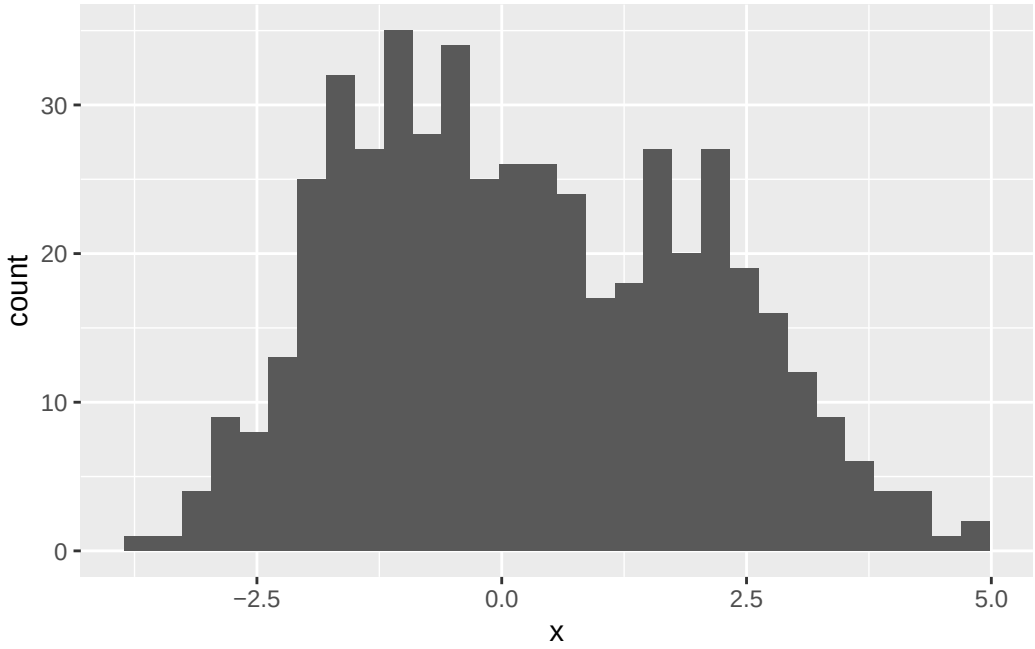
It is easier to maximise the log-likelihood:

$$l(\mu_1, \mu_2, \pi_1 | \mathbf{X}) = \sum_{i=1}^{n} \log [\pi_1 f_1(x_i) + (1 - \pi_1) f_2(x_i)]$$

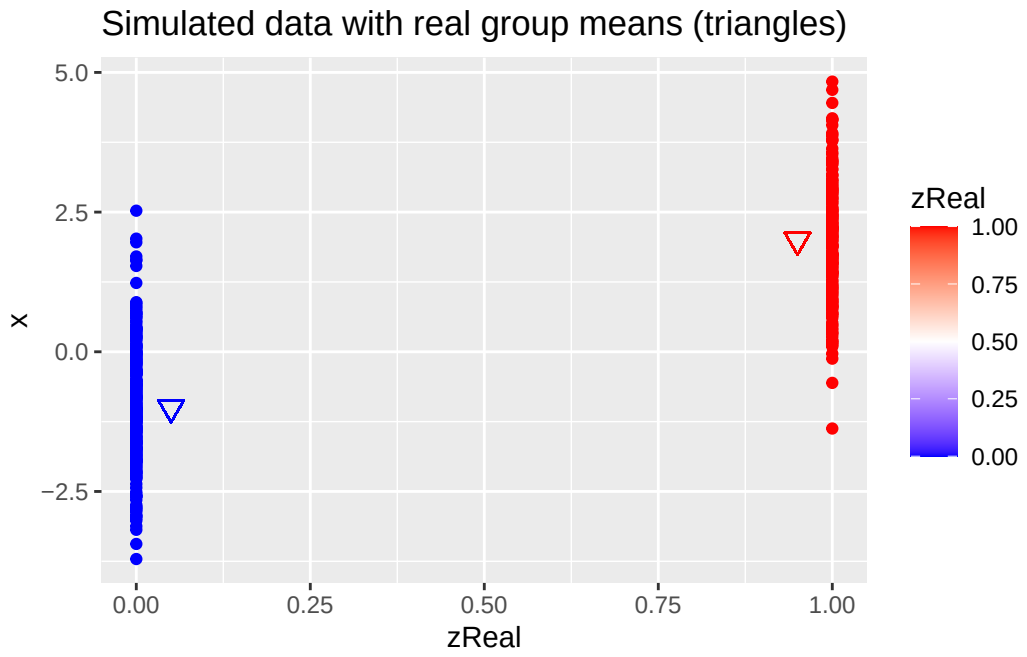
The optimisation is difficult because we have to take the log of a sum and the likelihood does not factorise further.

2.2 Simulate toy Data

```
library(tidyverse)
mu1Real <- 2
mu2Real <- -1
pi1Real = .4
set.seed(114)
zReal <- rbinom(500, size = 1, prob = pi1Real)
x <- ifelse(zReal==1, rnorm(500,mean=mu1Real), rnorm(500, mean=mu2Real))
data.frame(x) |>
  ggplot(aes(x = x)) +
  geom_histogram()
```




```
data.frame(x,zReal) |>
    ggplot(aes(zReal,x,color=zReal)) +
    geom_point() +
    scale_colour_gradient2(
      low = "blue",
      mid="white",
      high="red",
      midpoint = 0.5) +
      geom_point(x=.95,y=mu1Real, shape=25, col="red", size=3) +
      geom_point(x=0.05,y=mu2Real, shape=25, col="blue", size=3) +
      ggtitle("Simulated data with real group means (triangles)")
```



2.3 Parameter estimation

If we would know the cluster membership z_{i1}

$$z_{i1} = \begin{cases} 1 & \text{if } x_i \text{ belongs to group 1} \\ 0 & \text{if } x_i \text{ belongs to group 2} \end{cases}$$

with z_{i1} follows a Bernoulli distribution

$$B(\pi_1) = \pi_1^{z_{i1}}(1-\pi_1)^{(1-z_{i1})}$$

Then the density of x_i given z_{i1} becomes

$$f(x_i|z_{i1}) = f_1(x_i)^{z_{i1}} f_2(x_i)^{1-z_{i1}}$$

and the joint distribution of z_{i1} and x_i then becomes

$$\begin{aligned} f(x_i, z_{i1}) &= f(x_i|z_{i1})f(z_{i1}) && (1) \\ &= f_1(x_i)^{z_{i1}} f_2(x_i)^{1-z_{i1}} \pi_1^{z_{i1}}(1-\pi_1)^{(1-z_{i1})} && (2) \end{aligned}$$

and the log likelihood of the complete data becomes

$$l\left(\mu_1, \mu_2, \pi_1 | \mathbf{X}, \mathbf{Z}\right) = \sum_{i=1}^{n} z_{i1} \log\left[\pi_1 f_1(x_i)\right] + (1 - z_{i1}) \log\left[(1 - \pi_1) f_2(x_i)\right]$$

- Note, that in the notation of the Fraley and Raftery (1998) $z_2 = 1 - z_1$ and $\pi_2 = 1 - \pi_1$

2.3.1 EM algorithm

If we would know the cluster membership, we could estimate all model parameters based on the complete likelihood.

Note, that the complete likelihood also factorises nicely!

However, the cluster membership is unknown.

The EM algorithm has been developed for missing data problems.

It is an iterative algorithm that consists of two steps:

1. E-step: calculate the expected log likelihood given the data and the current model parameter estimates
2. M-step: maximise the expected log-likelihood
3. Iterate between 1 and 2 until convergence.

2.3.1.1 E-step

In iteration m+1

$$\begin{aligned}
Q\left(\mu_1, \mu_2, \pi_1 | \mathbf{X}, \mathbf{Z}\right) &= E\left[l\left(\mu_1, \mu_2, \pi_1 | \mathbf{X}, \mathbf{Z}\right) | \mathbf{X}, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m\right] \\
&= E\Big[\sum_{i=1}^{n} z_{i1} \log\left[\pi_1 f_1(x_i)\right] + (1 - z_{i1}) \log\left[(1 - \pi_1) f_2(x_i)\right] \\
&\quad \Big| \mathbf{X}, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m\Big] \\
&= \sum_{i=1}^{n} E\left[z_{i1} | x_i, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m\right] \log\left[\pi_1 f_1(x_i)\right] \\
&\quad + \sum_{i=1}^{n} \left(1 - E\left[z_{i1} | x_i, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m\right]\right) \log\left[(1 - \pi_1) f_2(x_i)\right]
\end{aligned}$$

Note, that the expected log likelihood simplifies to replacing the unknown class memberships in the complete likelihood by their expected values.

$$
\begin{aligned}
E\left[z_{i1}|\mathbf{X}, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m\right] &= 1 \times f(z_{i1} = 1|x_i, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m) \\
&\quad + 0 \times f(z_{i1} = 0|x_i, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m) \\
&= f(z_{i1} = 1|x_i, \mu_1 = \mu_1^m, \mu_2 = \mu_2^m, \pi_1 = \pi_1^m) \\
&= \frac{f(z = 1, x_i)}{f(x_i)} \\
&= \frac{\pi_1^m f_1(x_i)}{\pi_1^m f_1(x_i) + (1 - \pi_1^m) f_2(x_i)}
\end{aligned}
$$

We will refer to the expected class membership as $\hat{z}_{i1}^{m+1}$

2.3.1.2 M-step

Maximise the expected log-likelihood to obtain the unknown model parameters:

$$
\begin{aligned}
Q\left(\mu_1, \mu_2, \pi_1 | \mathbf{X}, \mathbf{Z}\right) = &\sum_{i=1}^{n} \hat{z}_{i1}^{m+1} \log \pi_1 + \sum_{i=1}^{n} \hat{z}_{i1}^{m+1} \log f_1(x_i) \\
&+ \sum_{i=1}^{n} \left(1 - \hat{z}_{i1}^{m+1}\right) \log(1 - \pi_1) + \sum_{i=1}^{n} \left(1 - \hat{z}_{i1}^{m+1}\right) \log f_2(x_i)
\end{aligned}
$$

So we observe that the expected log likelihood implies an estimation orthogonality between the normal distributions and the Bernoulli distribution.

So the parameter estimates that maximise the expected log-likelihood become:

$$
\begin{aligned}
\hat{\pi}_1^{m+1} &= \frac{\sum_{i=1}^{n} \hat{z}_{i1}^{m+1}}{n} \\
\hat{\mu}_1^{m+1} &= \frac{\sum_{i=1}^{n} \hat{z}_{i1}^{m+1} x_i}{\sum_{i=1}^{n} \hat{z}_{i1}^{m+1}} \\
\hat{\mu}_2^{m+1} &= \frac{\sum_{i=1}^{n} (1 - \hat{z}_{i1}^{m+1}) x_i}{\sum_{i=1}^{n} (1 - \hat{z}_{i1}^{m+1})}
\end{aligned}
$$

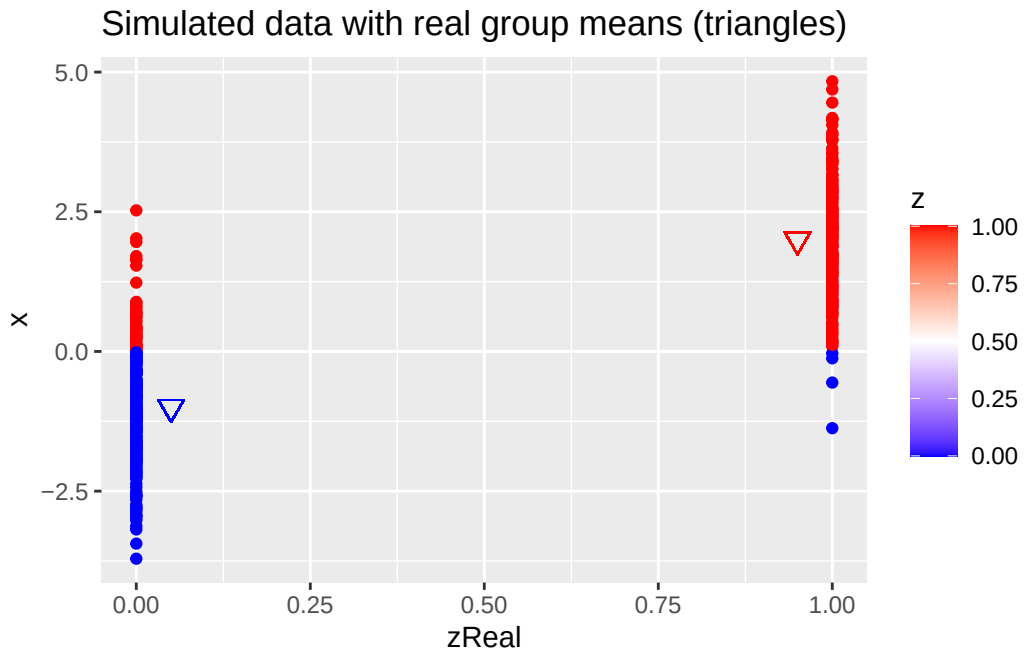
2.4 Example

2.4.1 Initialize

```r
z <- as.double(x > 0)
n <- length(z)

p <- data.frame(x,zReal,z) |>
  ggplot(aes(zReal,x,color=z)) +
  geom_point() +
  scale_colour_gradient2(
    low = "blue",
    mid="white",
    high="red",
    midpoint = 0.5) +
  geom_point(x=.95,y=mu1Real, shape=25, col="red", size=3) +
  geom_point(x=0.05,y=mu2Real, shape=25, col="blue", size=3) +
  ggtitle("Simulated data with real group means (triangles)")


p
```

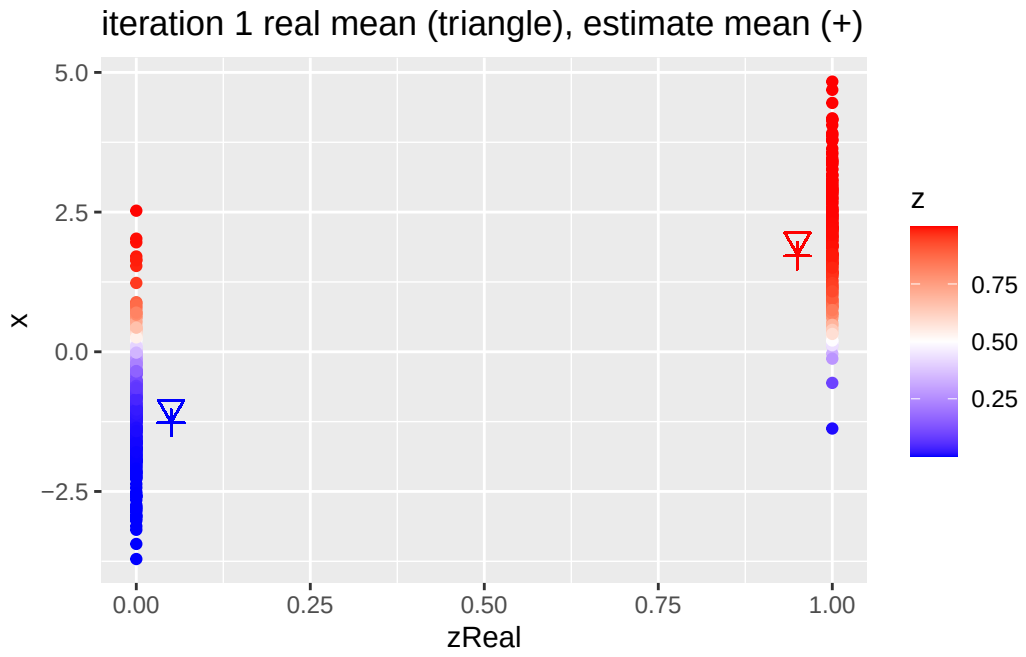


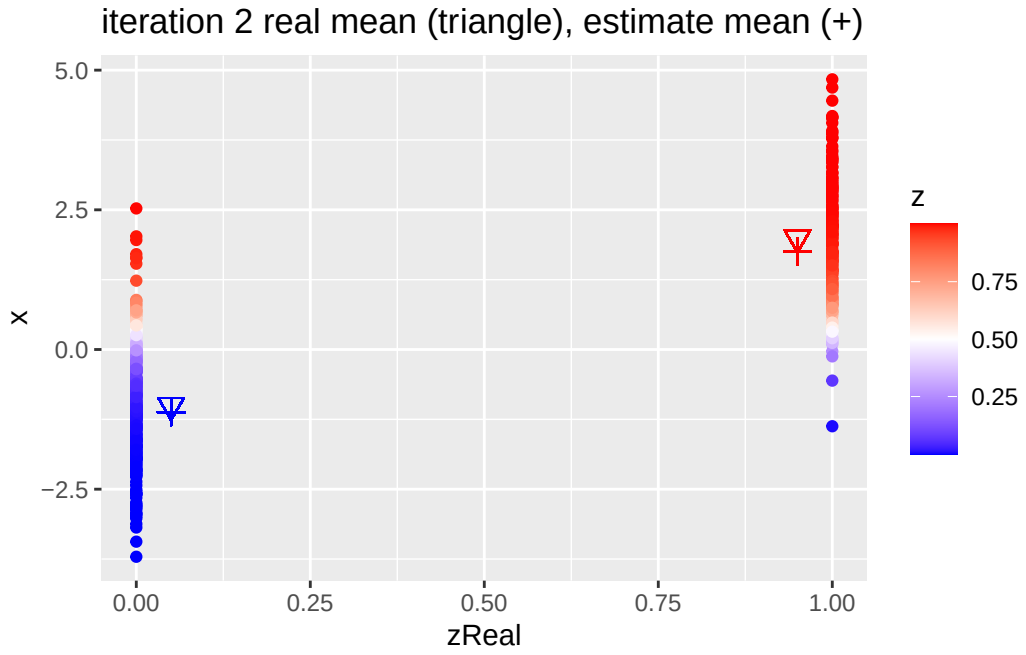
2.4.2 EM algorithm

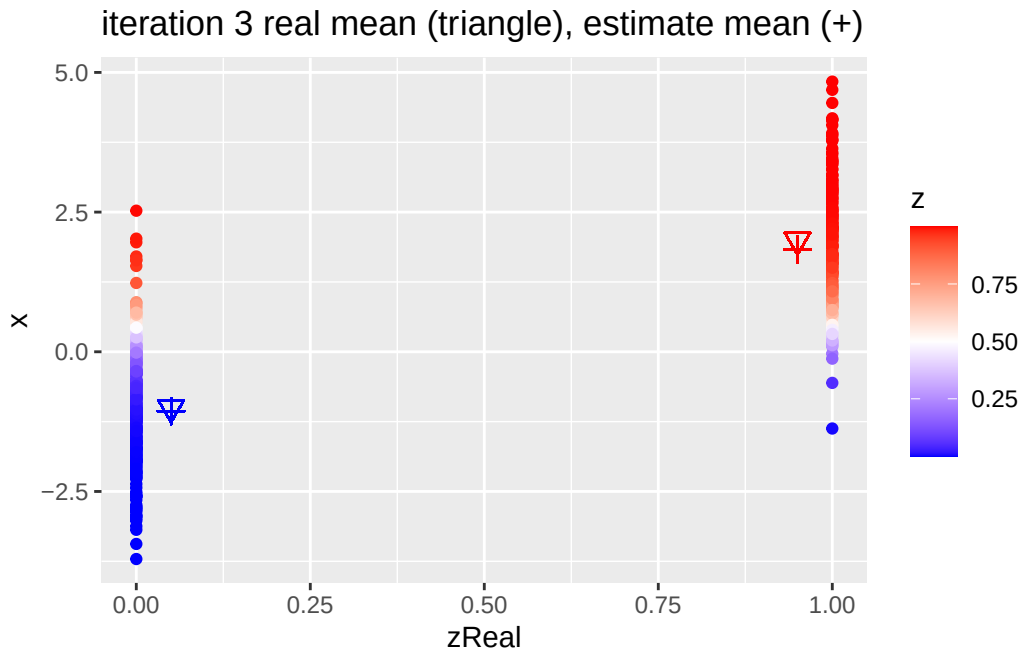
```
for (k in 1:10)
{
  ## M-step
  pi1 <- sum(z)/n
  mu1 <- sum(z * x)/sum(z)
  mu2 <- sum((1-z) * x)/sum(1-z)

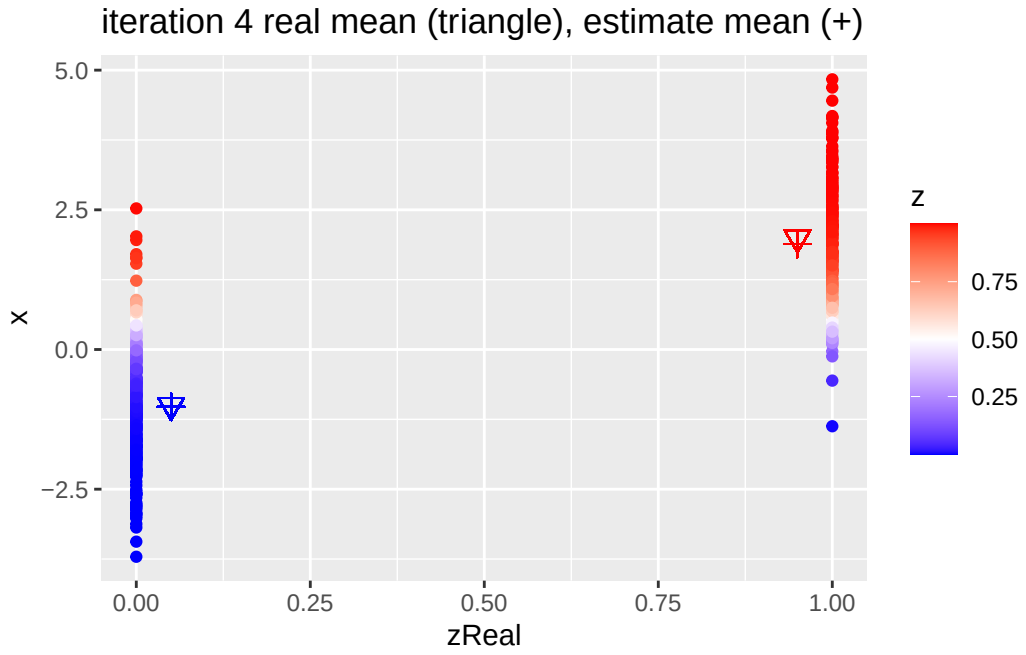
  ## E-step
  d1 <- dnorm(x,mean=mu1)
  d2 <- dnorm(x,mean=mu2)
  d <- pi1 * d1 + (1-pi1)*d2
  z <- pi1*d1/d
  p <- data.frame(x,zReal,z) |>
    ggplot(aes(zReal,x,color=z)) +
    geom_point() +
    scale_colour_gradient2(
      low = "blue",
      mid="white",
      high="red",
      midpoint = 0.5) +
      geom_point(x=.95,y=mu1Real, shape=25, col="red", size=3) +
      geom_point(x=0.05,y=mu2Real, shape=25, col="blue", size=3) +
      geom_point(x=.95,y=mu1, shape=3, col="red", size=3) +
      geom_point(x=.05,y=mu2, shape=3, col="blue", size=3) +
      ggtitle(paste0("iteration ",k, " real mean (triangle), estimate mean (+)"))

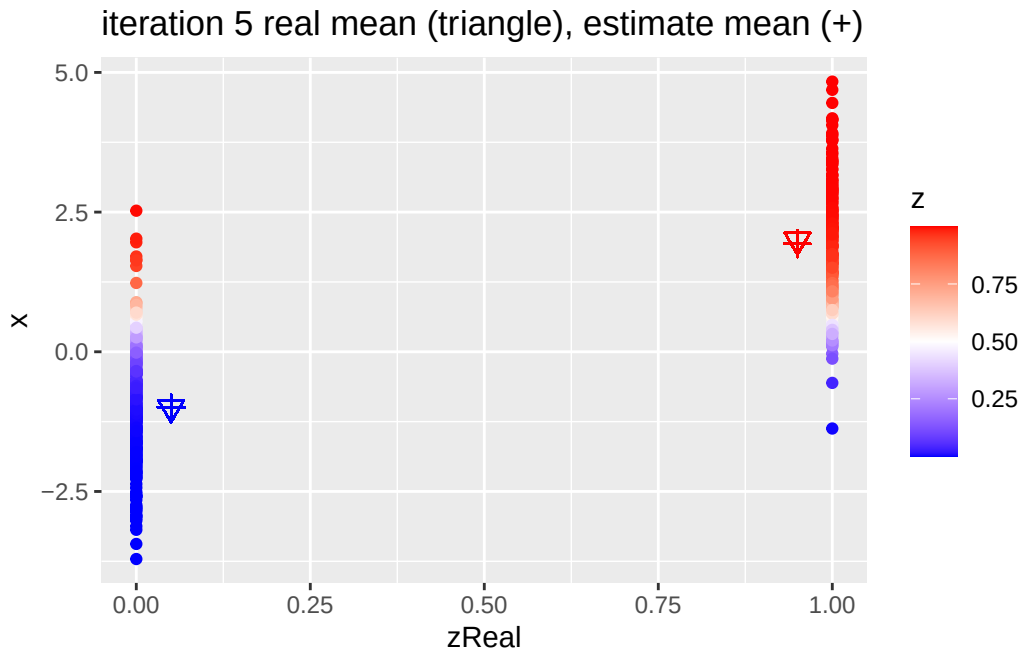
  print(p)
}
```

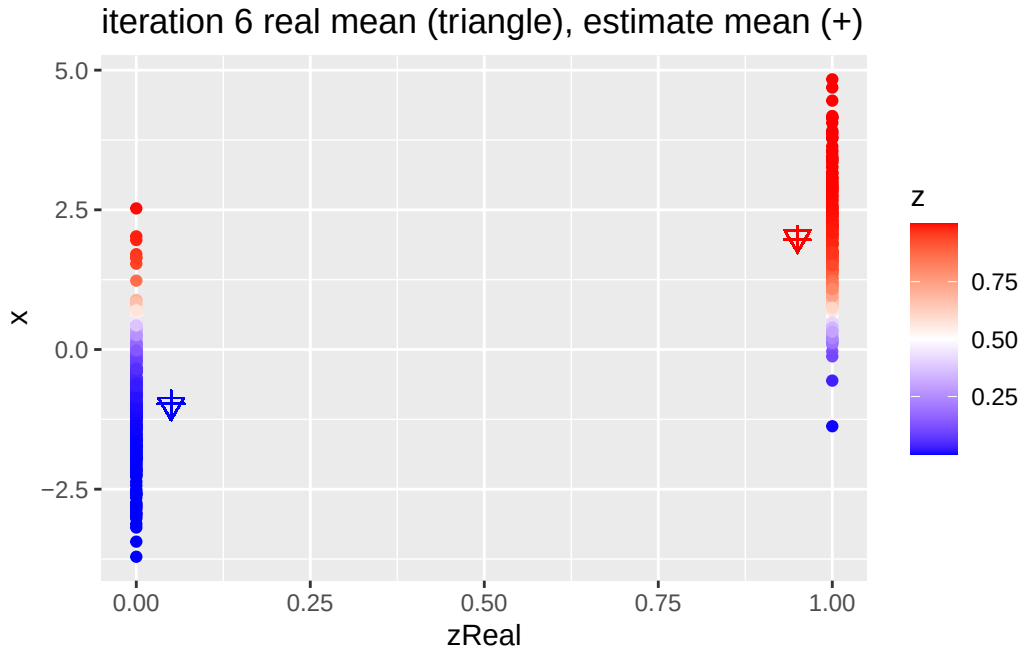

iteration 1 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25

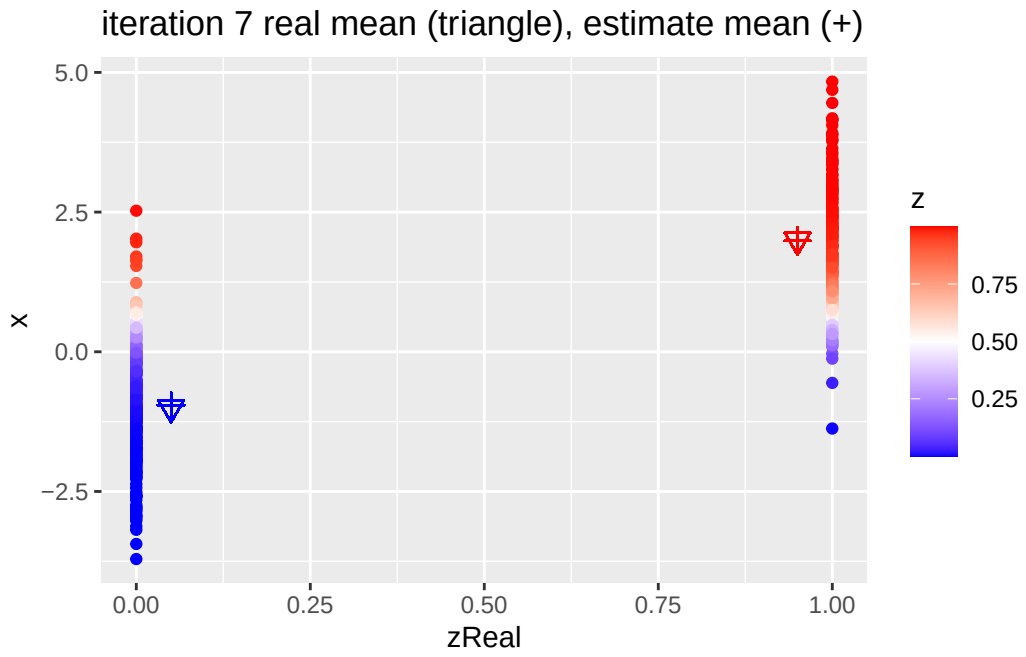

iteration 2 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25

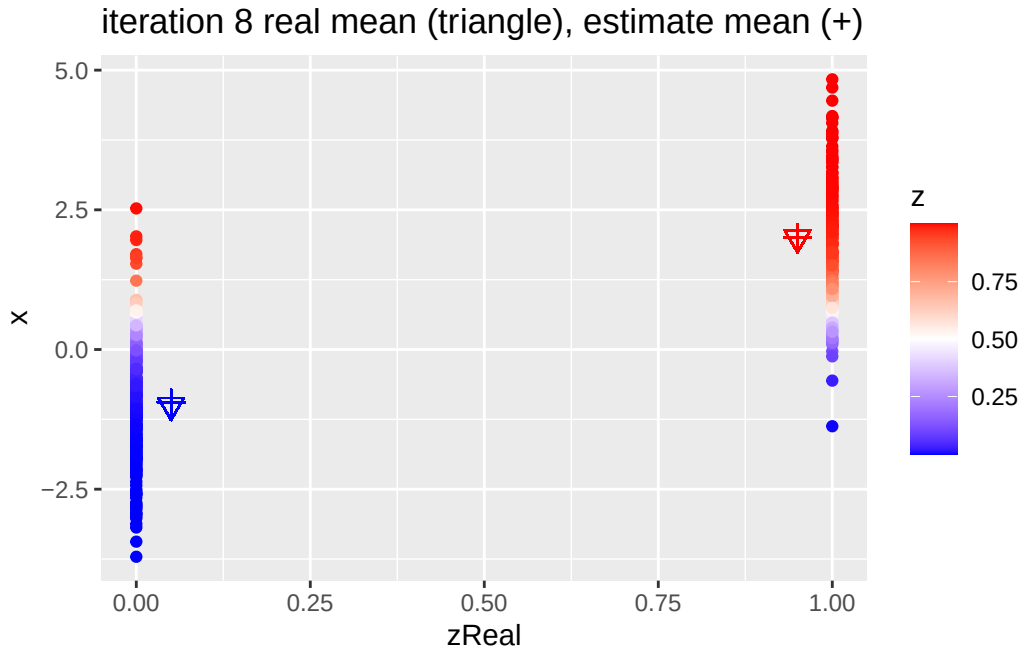

iteration 3 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25

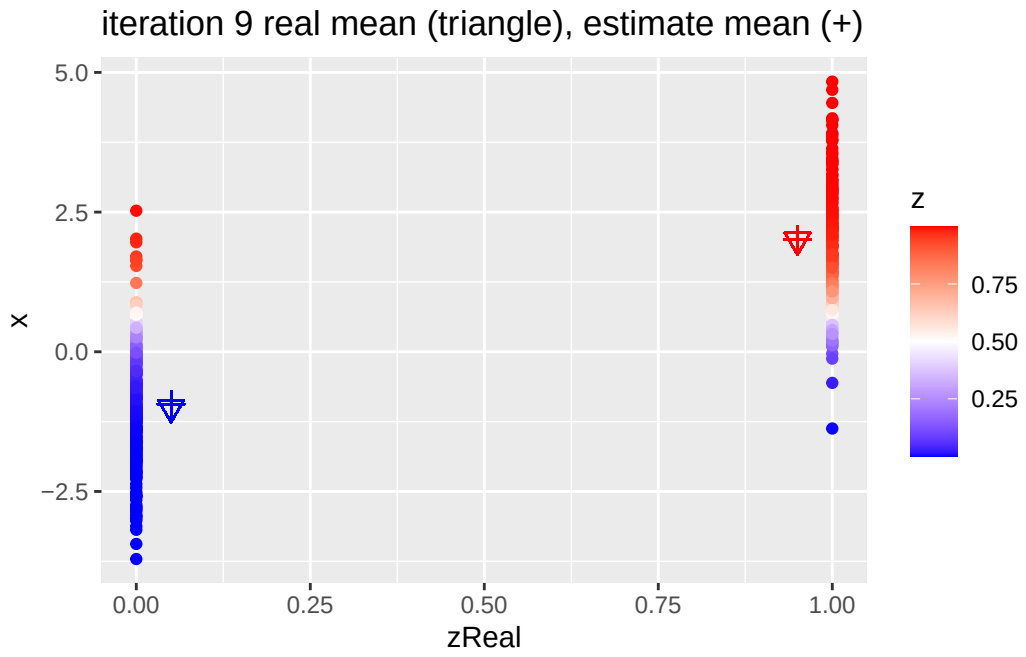

iteration 4 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25

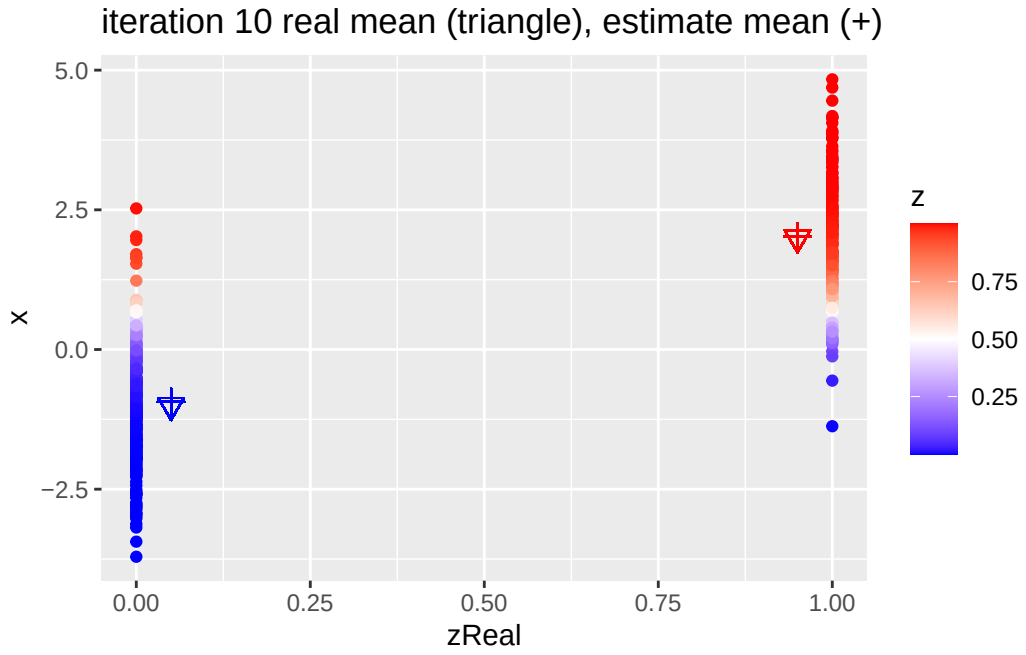

iteration 5 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25


iteration 6 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25


iteration 7 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25


iteration 8 real mean (triangle), estimate mean (+)
5.0
2.5
0.0
−2.5
x
0.00
0.25
0.50
0.75
1.00
zReal
z
0.75
0.50
0.25





2.4.2.1 Estimates

parameter	population	initial	estimate
mu1	2.0	1.715	2.020
mu2	-1.0	-1.270	-0.935
pi1	0.4	0.512	0.404

3 From EM to CEM: link with k-means

Fraley and Raftery (1998) state that

> "The standard k-means algorithm can be shown to be a version of the CEM algorithm corresponding to the uniform spherical Gaussian model $\Sigma_k = \lambda I$."

3.1 Soft versus hard assignment

- In EM, the E-step gives a *soft* assignment: observation i belongs to group 1 with probability $\hat{z}_{i1}$, e.g. 0.7, and to group 2 with probability $1 - \hat{z}_{i1}$, e.g. 0.3.
- The Classification EM (CEM) algorithm adds a C-step after the E-step that converts the soft assignment into a *hard* assignment: each observation is assigned to its most probable group.

$$\hat{z}_{i1}^C = \begin{cases} 1 & \text{if } \hat{z}_{i1} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- The M-step then uses ordinary (unweighted) means of the observations assigned to each group.

So, EM = soft assignment and CEM = hard assignment ("winner takes all").

3.2 CEM with spherical clusters with equal variance and equal proportions

Now assume the simplest possible model:

- all groups have the same variance λ in every direction: $\Sigma_k = \lambda I$ (in our toy example $\sigma^2 = 1$ for both groups);
- all groups have the same proportion, i.e. $\pi_1 = 1 - \pi_1 = 0.5$ ("uniform").

The posterior probability from the E-step then becomes

$$\hat{z}_{ik} \propto \exp\left(-\frac{\|x_i - \mu_k\|^2}{2\lambda}\right)$$

because the proportions and the variances are the same for all groups and cancel out.

Hence, the most probable group is the group with the nearest mean!

CEM with $\Sigma_k = \lambda I$ and equal π_k	k-means
E + C-step: assign each observation to its most probable group	Assign each observation to the nearest centroid
M-step: $\hat{\mu}_k$ is the mean of the observations assigned to group k	Recompute each centroid as the mean of its observations

This is exactly the same algorithm.

3.3 Implicit assumptions of k-means

K-means thus implicitly assumes that

- the clusters are spherical,
- the clusters have the same size and spread,
- the clusters contain approximately the same number of observations,
- the assignments are hard, i.e. there is no uncertainty on the cluster membership.

When these assumptions do not hold, e.g. for elongated clusters, clusters with a different spread or overlapping clusters, k-means performs poorly.

Model based clustering allows us to relax each of these assumptions: other covariance models (see Table 1 in Fraley and Raftery, 1998), EM instead of CEM and different proportions π_k.

3.4 Example

We implement CEM with equal proportions and equal variances for the toy data and start from the same initialisation as for EM.

```r
zCem <- as.double(x > 0)

for (k in 1:10)
{
  ## M-step
  mu1Cem <- sum(zCem * x)/sum(zCem)
  mu2Cem <- sum((1-zCem) * x)/sum(1-zCem)

  ## E-step with equal proportions and equal variances
  d1 <- dnorm(x, mean = mu1Cem)
  d2 <- dnorm(x, mean = mu2Cem)
  zSoft <- d1/(d1 + d2)

  ## C-step: hard assignment, i.e. assign to the nearest mean
  zCem <- as.double(zSoft > 0.5)
}
```

We compare the result with k-means, started from the same initial means.

```r
km <- kmeans(x, centers = c(mu1Init, mu2Init))

table(CEM = zCem, kmeans = km$cluster)
```

```
#>    kmeans
#> CEM   1   2
#>   0   0 291
#>   1 209   0
```

```r
knitr::kable(data.frame(
  parameter = c("mu1", "mu2"),
  population = c(mu1Real, mu2Real),
  EM = c(mu1, mu2) |> round(3),
  CEM = c(mu1Cem, mu2Cem) |> round(3),
  kmeans = km$centers[,1] |> round(3)
  )
  )
```

parameter	population	EM	CEM	kmeans
mu1	2	2.020	2.044	2.044
mu2	-1	-0.935	-1.024	-1.024

parameter	population	EM	CEM	kmeans

- CEM and k-means give the same partition and the same cluster means.
- Note, that CEM and k-means do not provide an estimate of π_1, nor the uncertainty on the cluster membership of each observation.
- In general, the hard assignment truncates the tails of overlapping groups and can bias the estimated cluster means.
- Here, the groups are fairly well separated, so all methods give estimates close to the population means.

Session info

Session info

```
#> [1] "2026-10-29 14:17:10 CET"
```

```
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24)
#>  os       macOS Tahoe 26.1
#>  system   aarch64, darwin23
#>  ui       X11
#>  language (EN)
#>  collate  nl_BE.UTF-8
#>  ctype    nl_BE.UTF-8
#>  tz       Europe/Brussels
#>  date     2026-10-29
#>  pandoc   3.10.2 @ /opt/homebrew/bin/ (via rmarkdown)
#>  quarto   1.9.37 @ /usr/local/bin/quarto
#>
#> - Packages -------------------------------------------------------------------
#>  package      * version date (UTC) lib source
#>  cli            3.6.6   2026-04-09 [1] CRAN (R 4.6.0)
#>  digest         0.6.39  2025-11-19 [1] CRAN (R 4.6.0)
#>  dplyr        * 1.2.1   2026-04-03 [1] CRAN (R 4.6.0)
#>  evaluate       1.0.5   2025-08-27 [1] CRAN (R 4.6.0)
#>  farver         2.1.2   2024-05-13 [1] CRAN (R 4.6.0)
#>  fastmap        1.2.0   2024-05-15 [1] CRAN (R 4.6.0)
#>  forcats      * 1.0.1   2025-09-25 [1] CRAN (R 4.6.0)
#>  generics       0.1.4   2025-05-09 [1] CRAN (R 4.6.0)
```

```
#>  ggplot2      * 4.0.3   2026-04-22 [1] CRAN (R 4.6.0)
#>  glue           1.8.1   2026-04-17 [1] CRAN (R 4.6.0)
#>  gtable         0.3.6   2024-10-25 [1] CRAN (R 4.6.0)
#>  hms            1.1.4   2025-10-17 [1] CRAN (R 4.6.0)
#>  htmltools      0.5.9   2025-12-04 [1] CRAN (R 4.6.0)
#>  jsonlite       2.0.0   2025-03-27 [1] CRAN (R 4.6.0)
#>  knitr          1.51    2025-12-20 [1] CRAN (R 4.6.0)
#>  labeling       0.4.3   2023-08-29 [1] CRAN (R 4.6.0)
#>  lifecycle      1.0.5   2026-01-08 [1] CRAN (R 4.6.0)
#>  lubridate    * 1.9.5   2026-02-04 [1] CRAN (R 4.6.0)
#>  magrittr       2.0.5   2026-04-04 [1] CRAN (R 4.6.0)
#>  mclust       * 6.1.3   2026-07-05 [1] CRAN (R 4.6.1)
#>  otel           0.2.0   2025-08-29 [1] CRAN (R 4.6.0)
#>  pillar         1.11.1  2025-09-17 [1] CRAN (R 4.6.0)
#>  pkgconfig      2.0.3   2019-09-22 [1] CRAN (R 4.6.0)
#>  purrr        * 1.2.2   2026-04-10 [1] CRAN (R 4.6.0)
#>  R6             2.6.1   2025-02-15 [1] CRAN (R 4.6.0)
#>  RColorBrewer   1.1-3   2022-04-03 [1] CRAN (R 4.6.0)
#>  readr        * 2.2.0   2026-02-19 [1] CRAN (R 4.6.0)
#>  rlang          1.2.0   2026-04-06 [1] CRAN (R 4.6.0)
#>  rmarkdown      2.31    2026-03-26 [1] CRAN (R 4.6.0)
#>  S7             0.2.2   2026-04-22 [1] CRAN (R 4.6.0)
#>  scales         1.4.0   2025-04-24 [1] CRAN (R 4.6.0)
#>  sessioninfo    1.2.4   2026-06-04 [1] CRAN (R 4.6.0)
#>  stringi        1.8.7   2025-03-27 [1] CRAN (R 4.6.0)
#>  stringr      * 1.6.0   2025-11-04 [1] CRAN (R 4.6.0)
#>  tibble       * 3.3.1   2026-01-11 [1] CRAN (R 4.6.0)
#>  tidyr        * 1.3.2   2025-12-19 [1] CRAN (R 4.6.0)
#>  tidyselect     1.2.1   2024-03-11 [1] CRAN (R 4.6.0)
#>  tidyverse    * 2.0.0   2023-02-22 [1] CRAN (R 4.6.0)
#>  timechange     0.4.0   2026-01-29 [1] CRAN (R 4.6.0)
#>  tzdb           0.5.0   2025-03-15 [1] CRAN (R 4.6.0)
#>  vctrs          0.7.3   2026-04-11 [1] CRAN (R 4.6.0)
#>  withr          3.0.3   2026-06-19 [1] CRAN (R 4.6.0)
#>  xfun           0.59    2026-06-19 [1] CRAN (R 4.6.0)
#>  yaml           2.3.12  2025-12-10 [1] CRAN (R 4.6.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
#>  * -- Packages attached to the search path.
#>
#> ------------------------------------------------------------------------------
```